

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	K. Ragavi	Reg. No.:	192524364
Section / Batch:	B. Tech - AI & DS	Date:	28/08/2026

Code of Conduct

I K. Ragavi (192524364) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: K. Ragavi

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

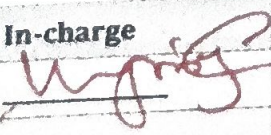
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. compare linear probing and separate chaining for collision handling in hashing based on :

- clustering effect
- memory usage
- performance under high load factor
- which method is more suitable for large-scale applications and why?

Linear Probing :

When a collision occurs, the next available position in the hash table is searched sequentially.

Separate chaining :

Each hash table position contains a linked list (chain) that stores all elements having the same hash index.

Basis	Linear Probing	Separate chaining
a) clustering effect	suffers from primary clustering, where consecutive occupied locations form groups.	clustering is avoided because collided elements are stored in linked lists.
b) memory usage	uses only the hash table array, so memory overhead is low.	Requires extra memory for linked-list nodes and pointers.
c) High load factor	Performance decreases rapidly when the table becomes highly occupied.	Handles higher load factor better because multiple elements can exist in one chain.

d) large-scale application	less suitable when the table becomes very full	more suitable because it handles collision and high load factors efficiently.
----------------------------	--	---

performance

- * Linear Probing has good cache performance because elements are stored continuously.
- * However, searching becomes slower as the load factor increases.
- * Separate chaining provides flexible storage and does not require an empty slot for every element.
- * Its performance mainly depends on the length of the chains.

conclusion

Separate chaining is generally more suitable for large-scale applications, especially when the number of elements changes frequently or the load factor is high. Linear Probing is preferable when memory overhead must be minimized and the hash table can be kept relatively empty.

2. compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirement (in-place vs extra space), cache behavior, and time complexity, which algorithm is preferable for linked lists and why? How does the answer change for arrays?

Quick sort	Merge sort	Basis
Average : $O(n \log n)$, Worst : $O(n^2)$	Best / Average / Worst : $O(n \log n)$	Time complexity
can be performed with low extra memory	Requires extra space for merging	Memory
Less suitable because Partitioning is difficult	Highly suitable because nodes can be split and merged easily	linked-list suitability
suitable for linked lists because sequential traversal is used	Good for arrays, but Poor for linked lists	cache behavior
usually not stable	stable	Stability
Risk of worst-case performance	Predictable $O(n \log n)$ performance	10 million nodes

Memory Requirements

Quick sort :

usually requires recursion stack space

Average stack space : $O(\log n)$

worst case : $O(n)$ if partitions are highly unbalanced

merge sort :

For linked lists, merging can be done by changing pointers.

Recursion requires approximately $O(\log n)$ stack space.

cache behaviour

For linked lists, random access is expensive. Quick sort requires partitioning and may involve jumping between nodes.

Merge sort processes the linked list sequentially, making it more suitable for linked-list memory access.

Which is preferable for linked lists?

Merge sort is preferable for linked lists

because:

- * It has guaranteed $O(n \log n)$ time.
- * It doesn't require random access.
- * Splitting and merging linked lists are efficient.
- * It can be implemented using node pointers.
- * It provides stable sorting.

What about arrays?

For arrays, quick sort is often preferable in practice because:

- * It has good cache locality.
- * It can be performed in-place.
- * It requires little extra memory.

Conclusion

Linked list - merge sort array $\rightarrow$ quick sort is often preferred, while merge sort is chosen when stability and guaranteed performance are important.

3 compare Quadratic Probing and Double Hashing in terms of

- a) clustering issues
- b) collision resolution efficiency
- c) suitability for dense hash tables

Quadratic Probing:

It resolves collisions by checking positions using quadratic increments such as $1^2, 2^2, 3^2, \dots$

Double Hashing:

It uses a second hash function to calculate the step size for finding the next position.

Basis	Quadratic Probing	Double Hashing
a) clustering issues	Reduces primary clustering but may have secondary clustering	Minimizes both primary and secondary clustering
b) collision resolution efficiency	Efficient for moderate load factors, but performance decreases as the table becomes full.	More efficient because elements are distributed more randomly.
c) Dense hash tables	Less suitable for highly filled tables.	More suitable for dense hash tables because it provides better distribution

Conclusion

Double Hashing is generally better than quadratic Probing because it reduces clustering and provides better collision resolution, especially when the hash table has a high load factor.

4. Compare best-case and worst case time complexity of sorting algorithms based on the following:

- Practical significance
- impact on algorithm selection

Sorting algorithm	Best case	worst case
Bubble sort	$O(n)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n^2)$
Heap sort	$O(n \log n)$	$O(n \log n)$

a) Best-case complexity shows how fast an algorithm can work for favorable input.

* Worst-case complexity shows the maximum time an algorithm may require.

- b)
- * Insert sort - suitable for small or nearly sorted data.
 - * Quick sort - suitable when fast average performance is required.
 - * Merge sort - suitable when guaranteed $O(n \log n)$ performance is needed.
 - * Heap sort - suitable when $O(n \log n)$ worst-case performance and low extra memory are required.

conclusion:

~~Best-case~~ analysis helps identify performance on favorable inputs, while worst-case analysis helps choose an algorithm that gives reliable performance. for large dataset, merge sort and Heap sort are preferred when predictable worst-case performance is important.

5 compare recursive and iterative implementations of quick sort in terms of:

- a) space complexity (stack usage)
- b) Ease of implementation
- c) Risk of stack overflow

Basic	Recursive quick sort	Iterative quick sort
a) space complexity	uses function call stack. Average : $O(\log n)$, worst : $O(n)$	uses an explicit stack. Average : $O(\log n)$, worst : $O(n)$
b) Ease of implementation	Easier to implementation and understand.	more complex because a stack must be managed manually.
c) Risk of stack overflow	Higher risk when partitions are highly unbalanced	Lower risk because it does not use recursive function calls.

Conclusion :

Recursive quick sort is simple and easy to understand, while iterative quick sort gives better control over stack usage and reduces the risk of recursion stack overflow.

Verifying